Sisteme de operare

Laborator 5

Semestrul I 2023-2024

Vlad Olaru

Laborator 5

- comunicare inter-procese (IPC)
- sisteme message passing
  - anonymous pipes
  - fifos (named pipes)
  - cozi de mesaje (System V IPC)

Pipe

- unidirectionale (half-duplex)
- nu pot fi folosite decat intre procese cu acelasi stramos
  - scenariu tipic: procesul parinte creaza un *pipe*, si apoi apeleaza *fork*

```
int pipe(int fd[2]);
```

- fd[0] deschis pentru citire, fd[1] pentru scriere
- iesirea lui fd[1] este intrarea lui fd[0]
- pt. citire/scriere din *pipe* se folosesc apelurile *read/write*

```
read(fd[0], buf, len);

write(fd[1], buf, len);
```

- dupa *fork*, copilul si parintele inchid unul dintre capetele de acces ale pipe-ului, in functie de directia de comunicare dorita
  - ex:

```
close(fd[0]);        // parinte
close(fd[1]);        // copil
```

=> directia de comunicare este parinte -> copil

Pipe (cont.)

- secventa de apel uzuala

```
#include <unistd.h>
int main(int argc, char* argv[]) {
        int fd[2];
        if(pipe(fd) < 0)
                perror("pipe"), exit(1);
        if(!fork())
                close(fd[1]);   // copilul inchide capatul sau de scriere
        else
                close(fd[0]);  // parintele inchide capatul sau de citire
}
```

Semantica operatiilor cu pipe-uri

- citirea dintr-un *pipe* gol blocheaza procesul
- scrierea intr-un *pipe* plin blocheaza procesul pana cand s-a citit suficient pentru ca scrierea sa se poata termina
- I/O non-blocant se seteaza cu ajutorul *fcntl*
- daca toate capetele de scriere ale unui *pipe* sunt inchise, *read* intoarce 0 octeti (EOF)
- daca toate capetele de citire ale unui *pipe* sunt inchise, *write* determina trimiterea unui semnal SIGPIPE catre procesul apelant
  - daca se ignora semnalul, *write* intoarce -1 cu errno = EPIPE
- nu se poate apela *lseek* pe un *pipe*
- scrierile mai mici de PIPE_BUF octeti sunt atomice, mai mult de PIPE_BUF pot fi interleaved cu scrierile altor procese

FIFOs (named pipes)

- *pipe* creaza un canal de comunicatie intre procese cu un stramos comun care a creat *pipe*-ul
- fisierele de tip FIFO permit IPC intre procese diferite
- apelul *stat/fstat* pe un fisier de tip FIFO indica in *st_mode* ca e fisier special (se poate folosi macro-ul S_ISFIFO)

```
int mkfifo(const char* pathname, mode_t mode);
```

- parametrul *mode* are aceeasi semnificatie ca la *open*, ex:

```
mkfifo("myfifo", 0644);
```

- dupa ce s-a creat fisierul FIFO, se deschide cu *open*, in mod uzual
- toate operatiile uzuale de I/O cu fisiere (*open*, *close*, *read*, *write*, *unlink*, etc) functioneaza si cu fisiere FIFO

FIFOs (cont.)

- la deschiderea unui FIFO
  - fara O_NONBLOCK: *open* cu O_RDONLY blocheaza procesul pana cand alt proces deschide FIFO-ul pt scriere; similar pt *open* cu O_WRONLY, procesul se blocheaza pana cand un alt proces deschide FIFO-ul pt citire
  - cu O_NONBLOCK: *open* cu O_RDONLY se intoarce imediat, in schimb cu O_WRONLY rezulta eroare cu errno = ENXIO daca nu exista nici un proces care a deschis FIFO pt citire
- citirile/scrierile au aceeasi semantica ca la *pipe*
- scrierile multiple concurente sunt atomice cata vreme scriu mai putin de PIPE_BUF octeti
- scenarii tipice de utilizare FIFOs
  - faciliteaza shell-ului evitarea folosirii fisierelor temporare cand transfera date de la un pipeline la altul
  - aplicatii client-server

mknod

- traditional, sistemele UNIX au creat FIFOs cu comanda/apelul sistem *mknod*
  - metoda generala de a crea fisiere in Unix:
    - fisiere obisnuite: S_IFREG
    - device-uri caracter S_IFCHR
    - device-uri block S_IFBLK
    - FIFOs S_IFIFO
    - Unix sockets S_IFSOCK

```
int mknod(const char* pathname, mode_t mode, dev_t dev);
```

- in particular, pt FIFO

```
mknod("myfifo", S_IFIFO | 0644, 0);
```

Arhitectura client-server cu FIFO

Probleme:

- cererile client trebuie sa fie mai mici de PIPE_BUF octeti pt. a fi citite atomic (fara intretesere intre scrierile clientilor)
- cum trimitem raspunsurile serverului catre fiecare client?
  - nu se poate folosi un singur FIFO pt toti clientii
  - un client nu stie cand sa-si citeasca propriul raspuns, si nu pe ale celorlalti client
- ambele probleme deriva din faptul ca nu exista canal de comunicatie separat intre client si server

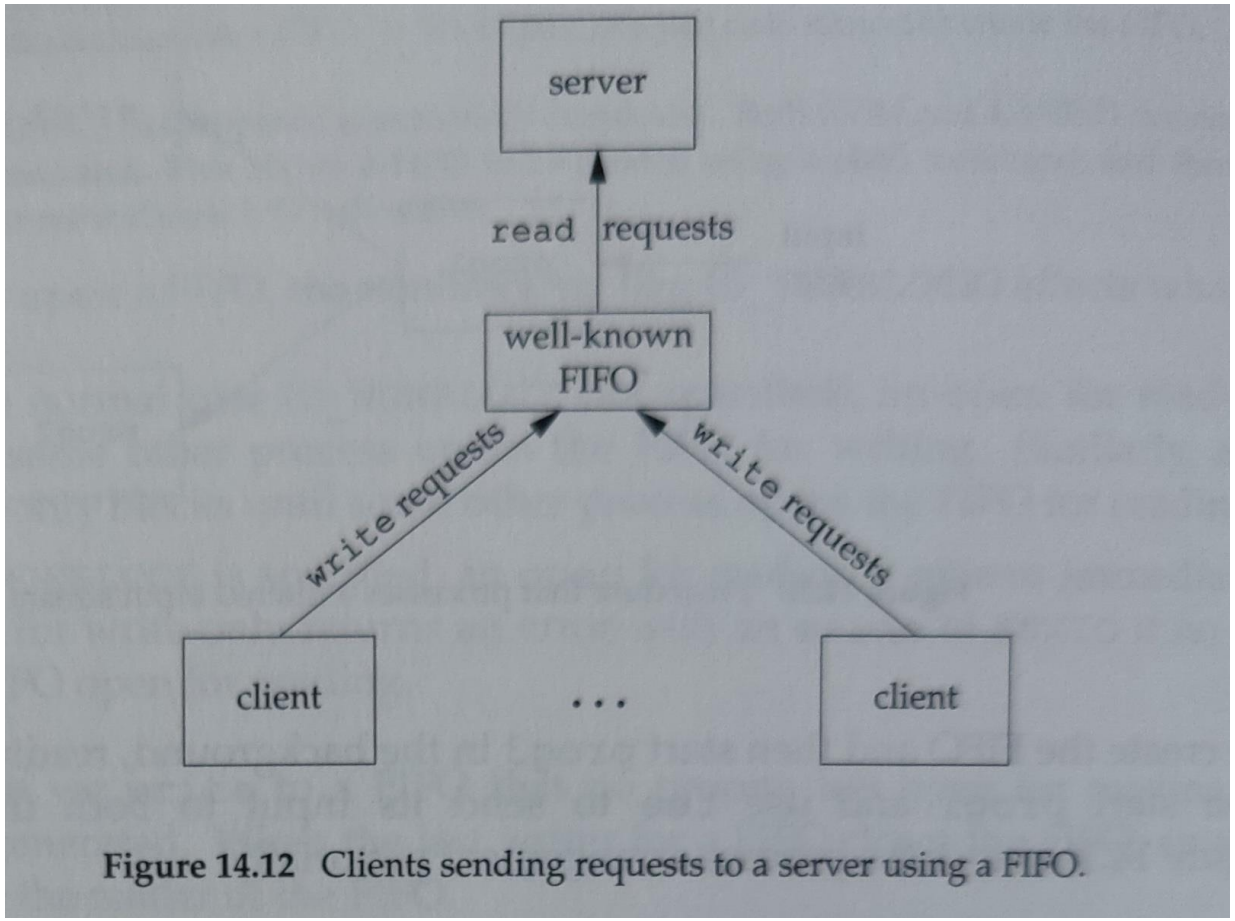


Figure 14.12 Clients sending requests to a server using a FIFO.

Comunicare client-server cu FIFO

Solutie posibila:

- fiecare client isi trimite cererea insotita de PID-ul sau
- serverul creeaza un FIFO de raspuns pt fiecare client folosind un nume care incorporeaza PID-ul client
  - ex: /tmp/client.XXXXX unde XXXXX = PID client

Probleme:

- clientul se defecteaza, serverul nu are cum sa afle => raman FIFO-uri in sistem
- serverul trebuie sa prinda SIGPIPE daca clientul termina inainte sa citeasca raspunsul serverului
- daca serverul deschide FIFO-ul RO, cand nr de clienti scade la 0, serverul citeste EOF
  - solutie: serverul deschide FIFO read-write

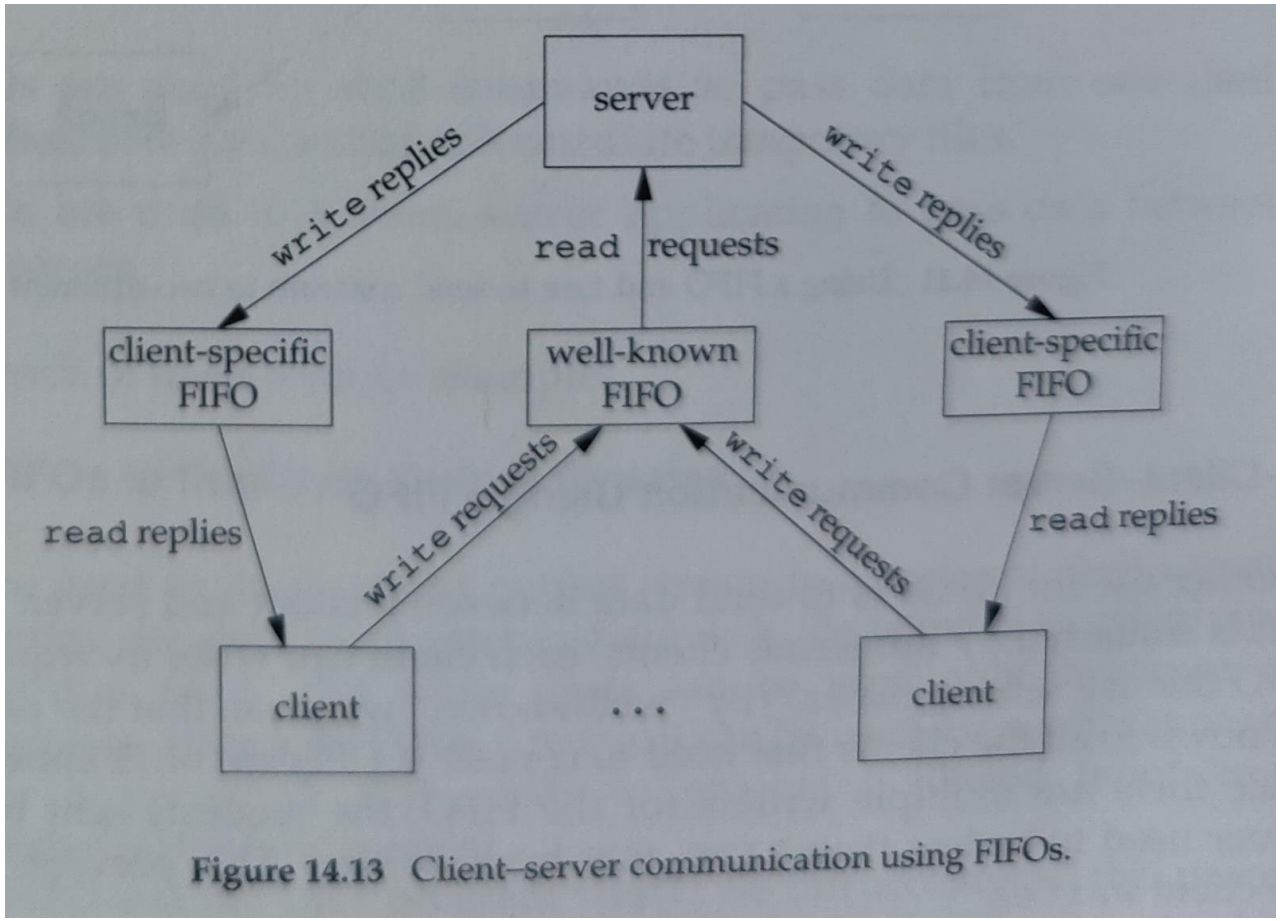


Figure 14.13 Client–server communication using FIFOs.

System V IPC

- cozi de mesaje, semafoare, memorie partajata
- toate aceste mecanisme folosesc un set comun de functii
- in kernel, toate structurile de date corespunzatoare sunt identificate printr-un ID (intreg nenegativ)
- pentru a folosi aceste mecanisme IPC e necesar sa stim acest ID
  - de ex: ca sa trimitem/primim mesaje intr-o/dintr-o coada de mesaje avem nevoie de ID-ul cozii
- in schimb, apelurile de creare a acestor mecanisme IPC necesita o cheie (o valoare de tip *key_t*) care va fi convertita de kernel intr-un ID

IPC rendez-vous

- pt. ca doua procese sa comunice prin mecanisme System V IPC trebuie sa poata identifica aceeasi structura IPC din kernel
- in acest sens, exista 3 posibilitati

1) un proces (desemnat uzual ca *server*) creeaza o noua structura IPC folosind o cheie de tip IPC_PRIVATE si stocheaza ID-ul returnat de kernel undeva unde il poate gasi alt proces care vrea sa se implice in IPC (*client*)

ex: in fisier, sau partajat intr-o paradigma *fork/exec*

2) serverul si clientul specifica o cheie intr-un header comun

Pb: daca exista in kernel o alta structura care foloseste aceeasi cheie, creearea noii structure IPC esueaza

IPC rendez-vous (cont.)

- pt. ca doua procese sa comunice prin mecanisme System V IPC trebuie sa poata identifica aceeasi structura IPC din kernel
- in acest sens, exista 3 posibilitati

...

3) serverul si clientul folosesc o cale comuna catre un fisier si un ID de proiect pe care le convertesc intr-o cheie cu ajutorul functiei *ftok*

- pentru a accesa o structura IPC existenta se foloseste cheia cu care a fost creata
- pentru a crea o structura IPC noua si a evita folosirea uneia existenta in apelurile de creeare a mecanismelor IPC se folosesc flag-urile IPC_CREAT | IPC_EXCL
- fiecare structura IPC din kernel are asociate permisiuni: UID/GID proprietar, UID/GID creator, modul de acces, nr de secventa, cheie

Structuri IPC, pros & cons

- system-wide, fara reference count
  - ex: o coada de mesaje din sistem care are mesaje in ea nu este stearsa din sistem cand procesul a terminat executia
  - e nevoie de apeluri sistem explicite (sau comenzi shell) pentru a o sterge
  - prin comparatie, pipe-urile dispar odata cu procesele care le utilizeaza (la FIFO ramane numele de fisier, dar datele sunt sterse)
- necunoscute sistemului de fisiere
  - e nevoie de comenzi si apeluri sistem speciale pentru a lucra cu ele
- DAR, sunt reliable, au flow control, orientate pe inregistrari, pot fi procesate si in alta ordine decat FIFO

Cozi de mesaje

- create cu *msgget*

int msgget(key_t key, int flag);

ex: flag IPC_CREAT |IPC_EXCL | 0644

- atributele lor (stocate intr-o structura *msqid_ds*) pot fi manipulate cu *msgctl*

int msgctl(int msgid, int cmd, struct msgid_ds *buf);

Valori *cmd*: IPC_STAT, citeste structura msgid_ds asociata cozii

IPC_SET, seteaza UID/GID, mode si dimensiunea cozii (dimensiunea cozii poate fi crescuta doar de root)

IPC_RMID, sterge imediat coada din sistem (si mesajele din ea; alte procese care folosesc ulterior coada primesc EIDRM)

Obs: pt SET si RMID e nevoie fie de drepturi de root fie ca

UID creator = UID proprietar

Cozi de mesaje (cont.)

- mesajele se scriu/citesc in/din coada cu *msgsnd*/*msgrcv*
- apelurile folosesc o structura definita de utilizator

```
struct msg {
        long mtype;                 // tip mesaj, valoare pozitiva
        char mtext[BUFSIZE];        // date mesaj
}

int msgsnd(int msgid, const void *ptr, size_t nbytes, int flag);

int msgrcv(int msgid, void *ptr, size_t nbytes, long type, int flag);
```

- pt. type = 0 la *msgrcv* se citeste primul mesaj din coada (politica FIFO), altfel se citeste primul mesaj din coada cu acel tip